

Árboles

Un enfoque muy útil y común para estudiar la teoría de grafos es restringir su abordaje a un tipo particular; por ejemplo, se podría enfocar en comprender sólo los ciclos, los grafos completos o las ruedas, en lugar de abarcar todos los grafos en general. Por ello, a continuación se presentan sólo los de tipo árboles. Se espera que al final de esta sección se tenga una mejor comprensión de esta clase de grafo, así como su relevancia y el por qué merece su propia sección.

Los árboles son, particularmente, útiles en informática, donde se emplean en una amplia gama de algoritmos. Por ejemplo, los árboles se utilizan para construir algoritmos eficientes para localizar elementos en listas. Se pueden usar en algoritmos, como la codificación de Huffman, que construyen códigos eficientes ahorro de costes en la transmisión y almacenamiento de datos; también se pueden usar árboles para estudiar juegos como damas y ajedrez y ayudan a determinar las estrategias ganadoras; para modelar procedimientos realizados utilizando una secuencia de decisiones. La construcción de estos modelos puede ayudar a determinar la complejidad computacional de los algoritmos basados en una secuencia de decisiones, como los de clasificación.

Definición



Definición

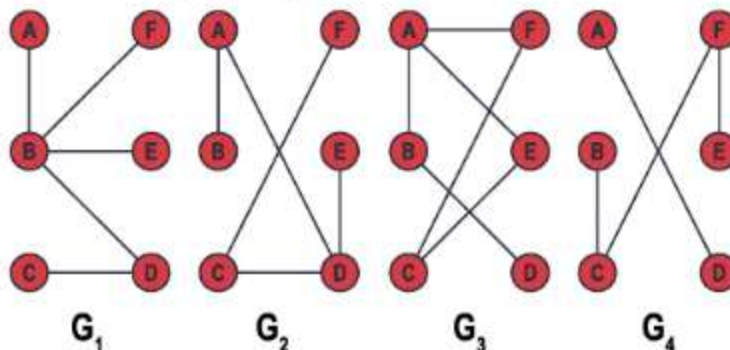
Un árbol es un grafo simple conectado que no tiene ciclos.

En el siguiente ejemplo se pueden ver varios grafos para identificar cuál de ellos es un árbol.



Ejemplo

Identifique cuáles de los siguientes grafos son árboles:



Se puede notar que los grafos G_1 y G_2 son árboles, pues son conectados y no tienen ciclos. G_3 no es un árbol, debido a que tiene un ciclo $AECFA$, y el grafo G_4 no es un árbol, ya que no es conectado.

Teoremas

Los siguientes teoremas dan algunas propiedades de los árboles:

Teorema

Un grafo T es un árbol si y sólo si entre cada par de vértices distintos de T existe un camino único.

El grafo es conectado, por lo tanto, existe el camino y la unicidad se garantiza, ya que no existen ciclos.

Se dice que un grafo es **un bosque** si no contiene ciclos. En otras palabras, la desconexión de grafos que no tienen ciclos forma un conjunto de árboles que se llaman bosque.

Corolario

Un gráfico B es un bosque si y solo si entre cualquier par de vértices de B hay, a lo sumo, un camino.

Recuerde que la palabra **a lo sumo** significa que hay un camino o que no hay ninguno, por lo que un bosque puede tener la desconexión, pero cada uno de sus elementos debe ser un árbol.

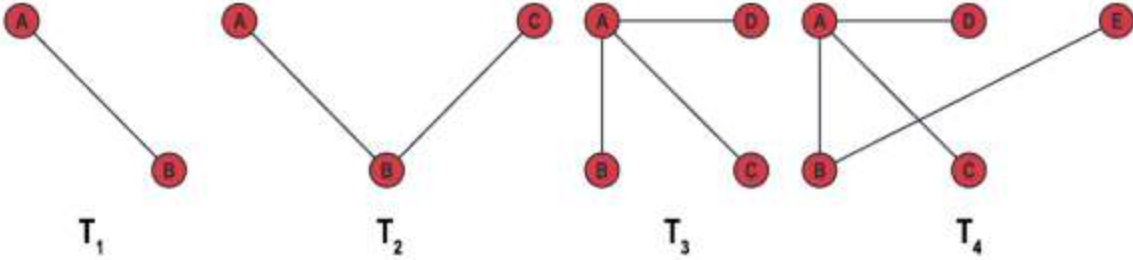
Otra propiedad bien interesante y que define el concepto de **hoja** es:

Teorema

Cualquier árbol con, al menos, dos vértices tiene, al menos, dos vértices de grado uno.

Como el grafo no debe tener ciclos, entonces deben existir, mínimo, dos vértices que solo son adyacentes, solo con uno de los otros vértices.

Note que los siguientes árboles cumplen con la propiedad:



Se puede ver que en T_1 se cumple el teorema de inmediato; T_2 se puso el máximo de vértices que se pueden poner para que se cumpla la propiedad de que es un árbol, se debe tener en cuenta que esos dos lados puede ser entre cualquier par de vertices, pero solo deben ser dos, y se cumple el teorema: tiene dos hojas; para el grafo T_3 , esos tres lados se pueden poner en configuraciones diferentes, sin formar ciclos, por lo tanto, tiene tres vértices de grado 1. Por último, el grafo T_4 es un árbol que también tiene por lo menos 2 vértices de grado 1.

El siguiente teorema habla de la cantidad de lados que hay en un árbol.

Teorema

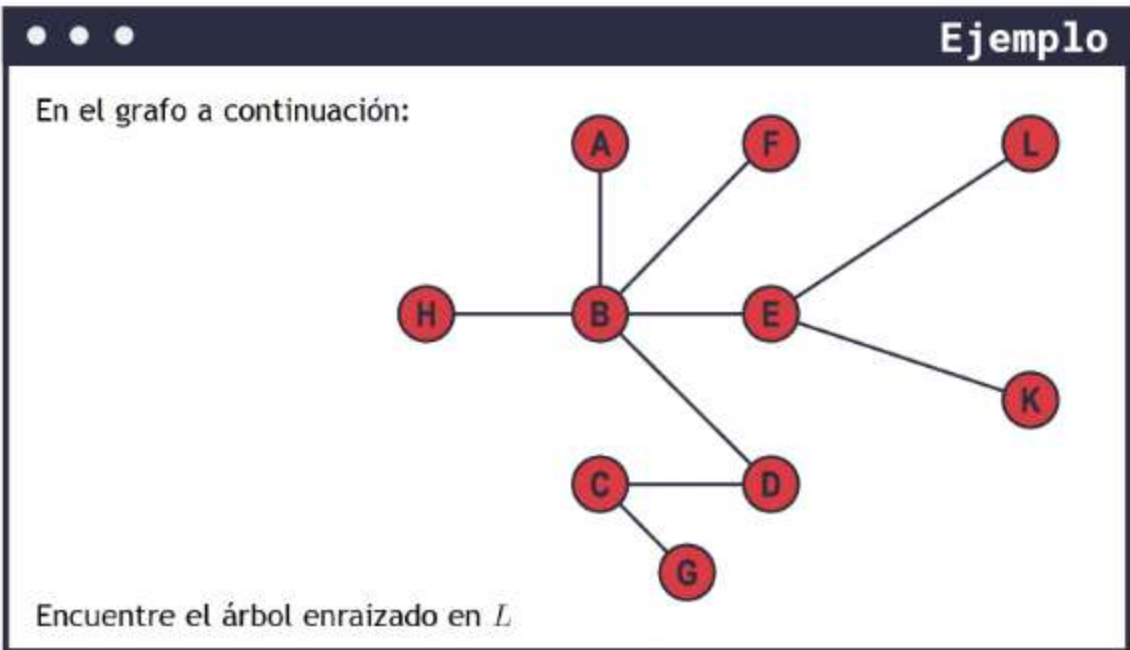
Un árbol T con v vértices y e lados cumple que $e = v - 1$

En el ejemplo anterior, note que T_1 tiene dos vértices, lo que significa que solo tiene un lado. Además, T_4 tiene 5 vértices, lo que significa que el árbol tiene 4 lados.

Árboles enraizados

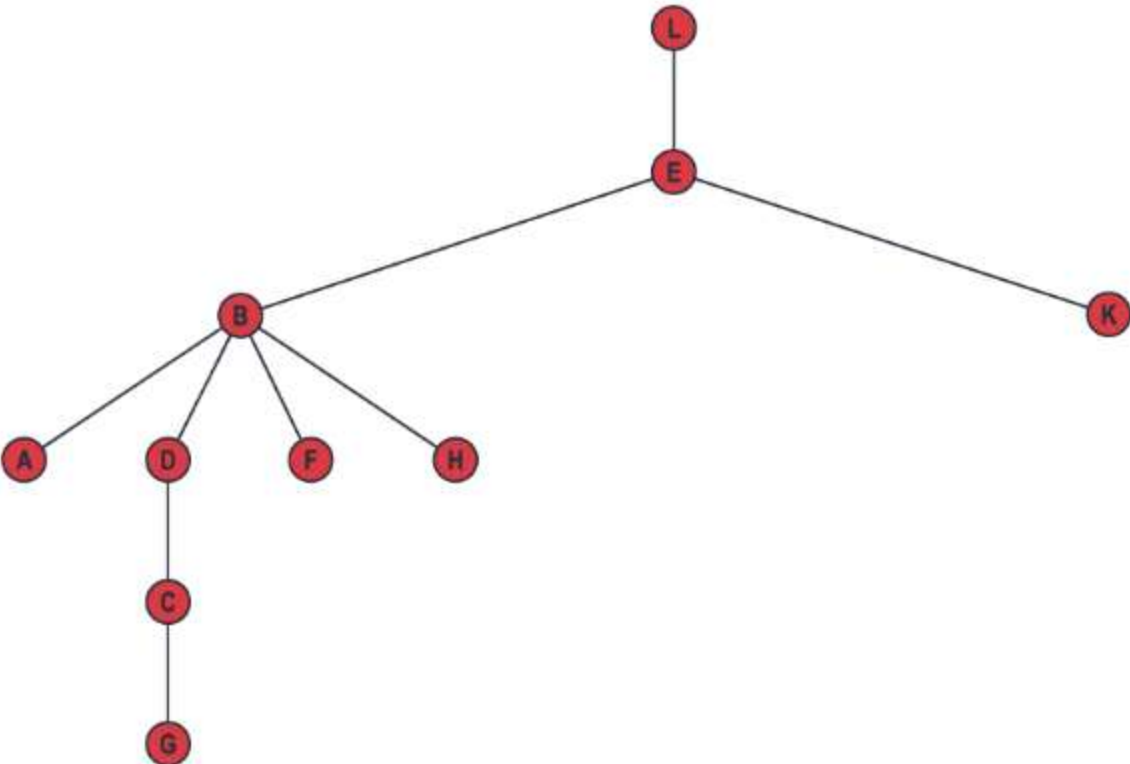
Hasta ahora se han tratado a los árboles como un tipo de grafo simple con cierta característica, sin embargo, es importante considerar algunas propiedades adicionales que pueden ser útiles para solucionar problemas.

Para hablar de árboles enraizados se puede partir de cualquier árbol, como los vistos anteriormente, y se le asigna a cualquiera de sus vértices un nombre, conocido como la **raíz**. Todos los demás vértices tienen una posición relativa con respecto a este, pues existe un camino único desde él hasta la raíz. Entonces, desde cualquier vértice, se viaja de regreso a la raíz, exactamente de una manera, lo que permite distinguir la relación del primero con la segunda.

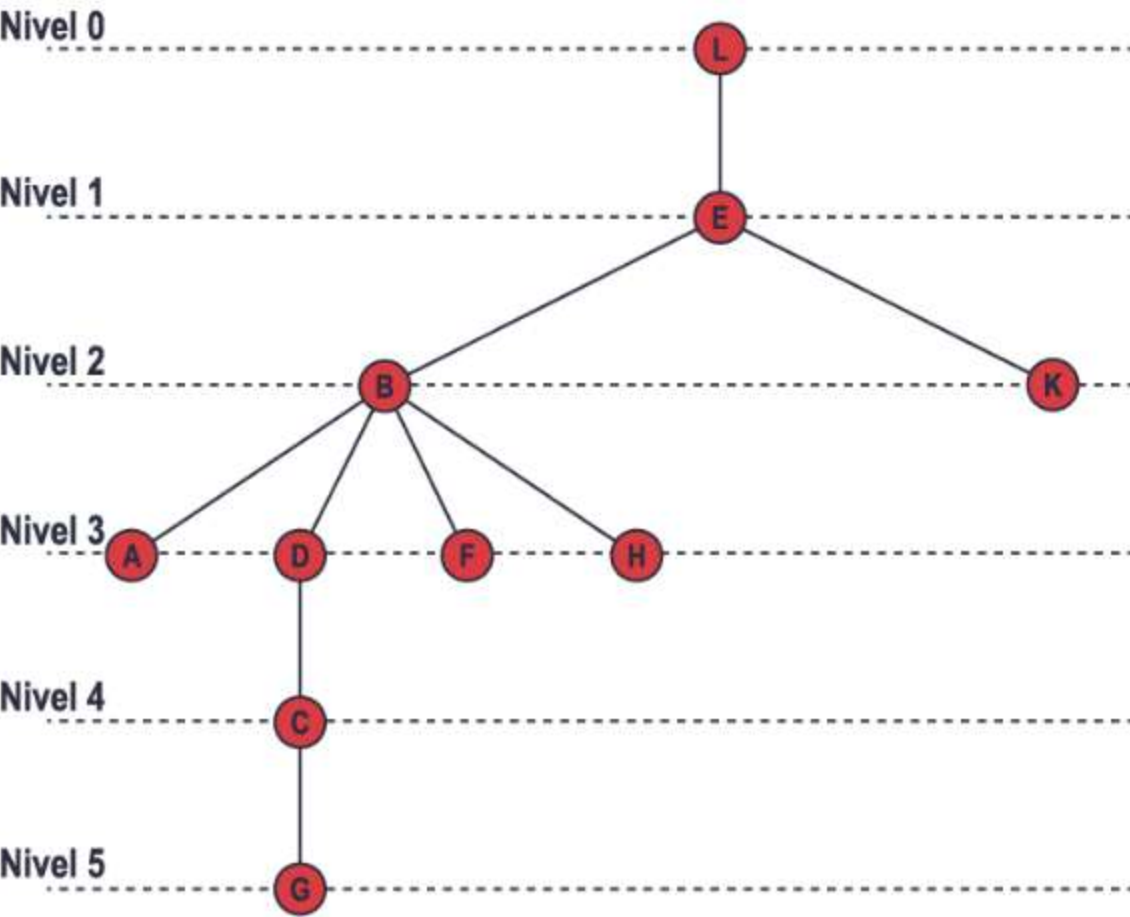


Este árbol se puede poner con *L* en la parte superior y los demás vértices se pueden poner en forma decreciente según su adyacencia. Dos vértices adyacentes, dependiendo de la raíz, se conocen como **padre**, y el vértice superior se conoce como **hijo**. A los vértices que comparten un mismo padre se les llama **hermanos**. Esto también es posible hacerlo de forma creciente, donde la raíz queda en la parte inferior, los padres, son los que estan abajo y los hijos arriba. Aquí se pondrán los padres en la parte superior.

El árbol enraizado se puede representar de la siguiente manera:

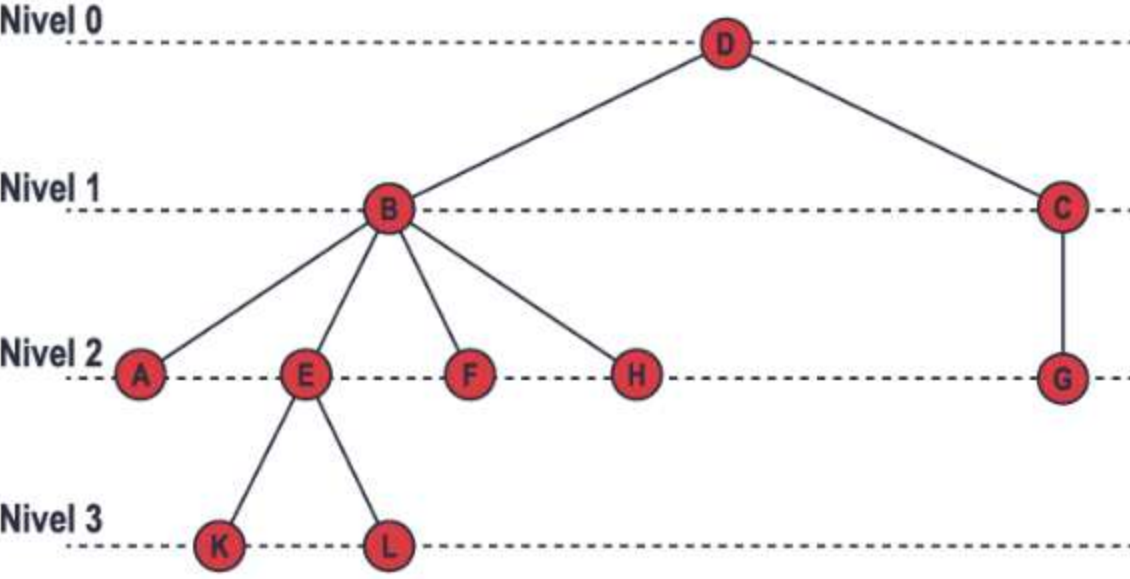


Para ser claros en la construcción del árbol enraizado, se quiere que los hermanos queden al mismo nivel en el gráfico. Se conoce el nivel de profundidad que hay desde la raíz hasta cada uno de los vértices, siendo la raíz el nivel cero.



En el ejemplo con un árbol enraizado en *L*, el vértice con más alto nivel es *G*, 5. El nivel más alto es la **altura** del árbol enraizado que, en este caso, es 5.

El árbol del ejemplo está enraizado en *L*, pero, ¿qué pasa si el árbol se enraiza en *D*? En este caso el árbol queda así:



La altura del árbol enraizado en *D* es menor que la del enraizado en *L*. Además, algunos padres pasan a ser hijos o algunos hermanos pasan a ser padres.

Cuando se enraiza un árbol, lo que se busca que este tenga el mayor equilibrio posible. En el último ejemplo se nota que las hojas quedan entre los niveles 2 y 3, lo que hace que haya un balance.

Se dice que un árbol enraizado de altura *n* está balanceado o equilibrado si todas sus hojas se ubican en los niveles *n – 1* y *n*; según esto el último ejemplo representa árbol enraizado balanceado.

Ya se sabe que en un árbol los vértices de grado 1 son hojas y que a los demás se conocen como vértices internos. En un árbol enraizado todos los vértices internos son padres, sin embargo, note que la cantidad de hijos que tiene cada padre no debe ser siempre la misma.

Definición



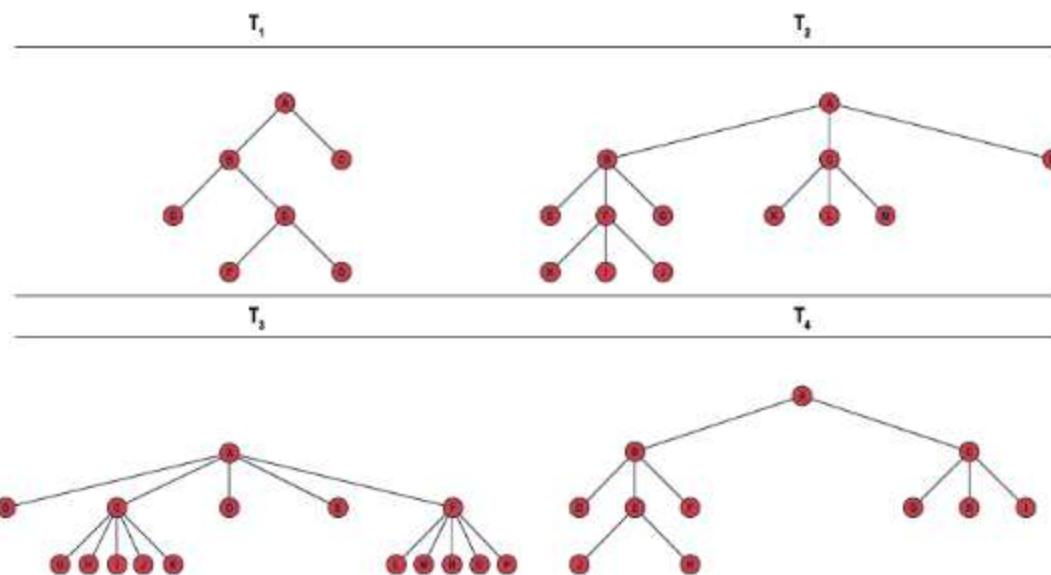
Definición

Un árbol enraizado se llama m -ario si cada vértice interno no tiene más de m hijos.
Un árbol se llama un árbol m -ario completo si cada vértice interno tiene exactamente m hijos. Un árbol m -ario con $m = 2$ se llama un árbol binario.



Ejemplo

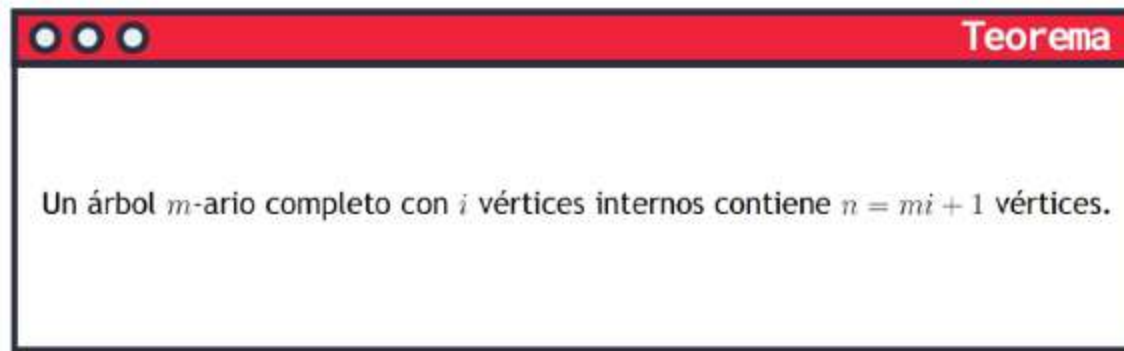
Los árboles a continuación son árboles enraizados m -arios:



Note que el árbol T_1 es binario, el grafo T_2 es un árbol 3-ario, el árbol T_3 es un árbol 5-ario y el T_4 es un árbol 3-ario, debido a que el máximo de hijos que tiene un vértice interno es 3. Pero los m -arios completos solo son T_1 , T_2 y T_3 , pues los que son vértices internos siempre tienen la misma cantidad de hijos.

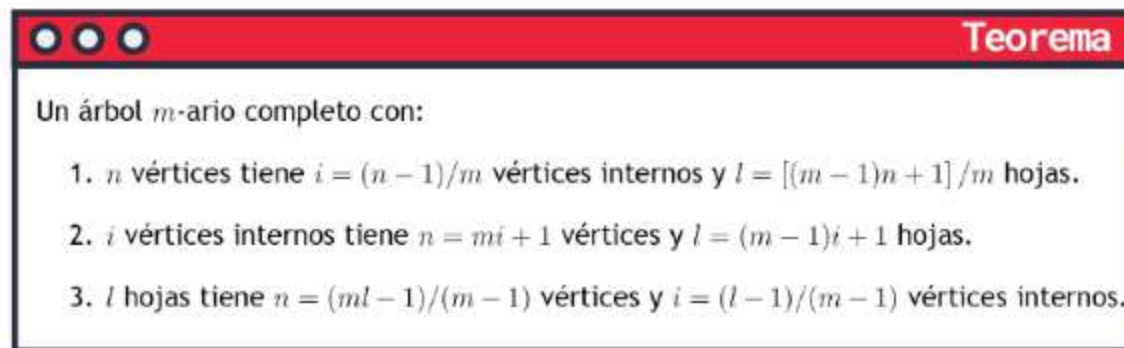
Como algunas propiedades de estos tipos de árboles

Teoremas



En el ejemplo anterior, el árbol T_1 que es binario completo, tiene 3 vértices internos y por lo tanto tiene $n = 2 \cdot 3 + 1 = 7$ vértices. El árbol T_2 que es 3-ario completo, tiene 4 vértices internos, por lo tanto contiene $n = 3 \cdot 4 + 1 = 13$ vértices.

Además de la propiedad anterior, existen estas otras 3 que permiten conocer la cantidad de vértices, vértices internos o hijos que tiene un árbol m -ario completo.



Así, si hay un árbol binario completo con 7 hojas, por el numeral 3, se tiene que el árbol tiene $n = (2 \cdot 7 - 1)/(1) = 13$ vértices y el número de vértices internos es $i = (7 - 1)/1 = 6$. Si dos personas diferentes dibujan el grafo, puede ser que sus dibujos queden diferentes, pues se pueden conseguir de muchas maneras los 6 vértices internos.

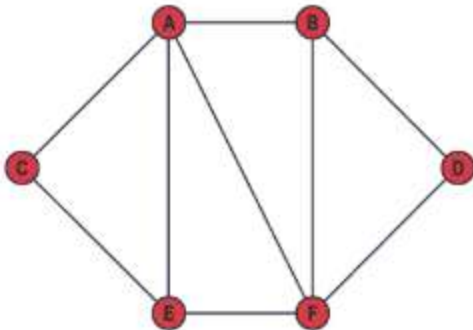
Ahora, si el número de vértices es 9 y se tiene un grafo 4-ario completo por el numeral 1, el número de vértices internos es $i = (9 - 1)/4 = 2$ y el número de hojas es $l = [(4 - 1)9 + 1]/4 = 7$.

Dentro de las aplicaciones de los árboles enraizados binarios, están: los árboles de búsqueda binaria, árboles de decisión, los juegos de árboles, entre otros.

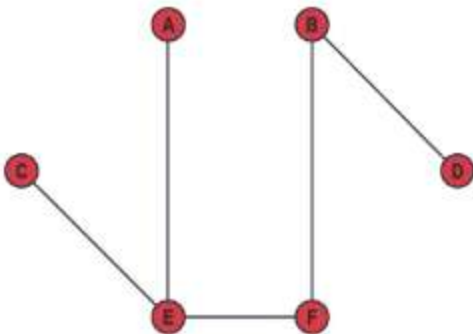
Árboles de expansión

Un árbol de expansión de un grafo no dirigido es un subgrafo que incluye todos los vértices del original y forma un árbol. En otras palabras, es una forma de quitar algunos lados de un grafo para crear uno más pequeño y simple que aún contenga todos los vértices originales, que sea conectado y no contenga ciclos.

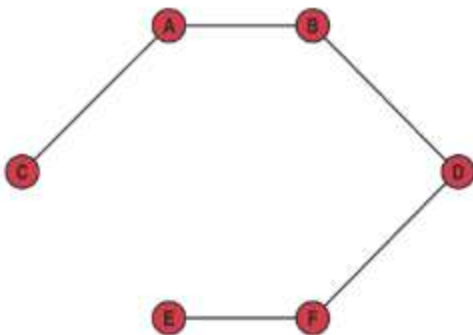
Cuando se habla de árboles de expansión es importante tener varias propiedades presentes, como es la minimalidad, es decir, que contenga el mínimo número posible de lados.



El cual es un grafo simple, puede tener variós árboles de expansión, uno de ellos es:



Pero también puede ser:



Y se pueden encontrar más, pero la idea es que con los lados y los vértices del grafo dado se encuentre un árbol que satisfaga las propiedades de un árbol de expansión.

El siguiente teorema da una propiedad muy importante de los grafos conectados:

Teorema

Un grafo es conectado si y solo si tiene un árbol de expansión.

El grafo del ejemplo anterior es conectado por que tiene un árbol de expansión. Es claro que si se tiene un árbol de expansión se puede obtener cualquier grafo con la misma cantidad de vértices, solo se le deben agregar los lados para conseguir el deseado.

La busqueda del árbol de expansión se ha usado para grafos con pesos y para encontrar el árbol de expansión que contenga la menor suma de pesos.

Algoritmo de Kruskal

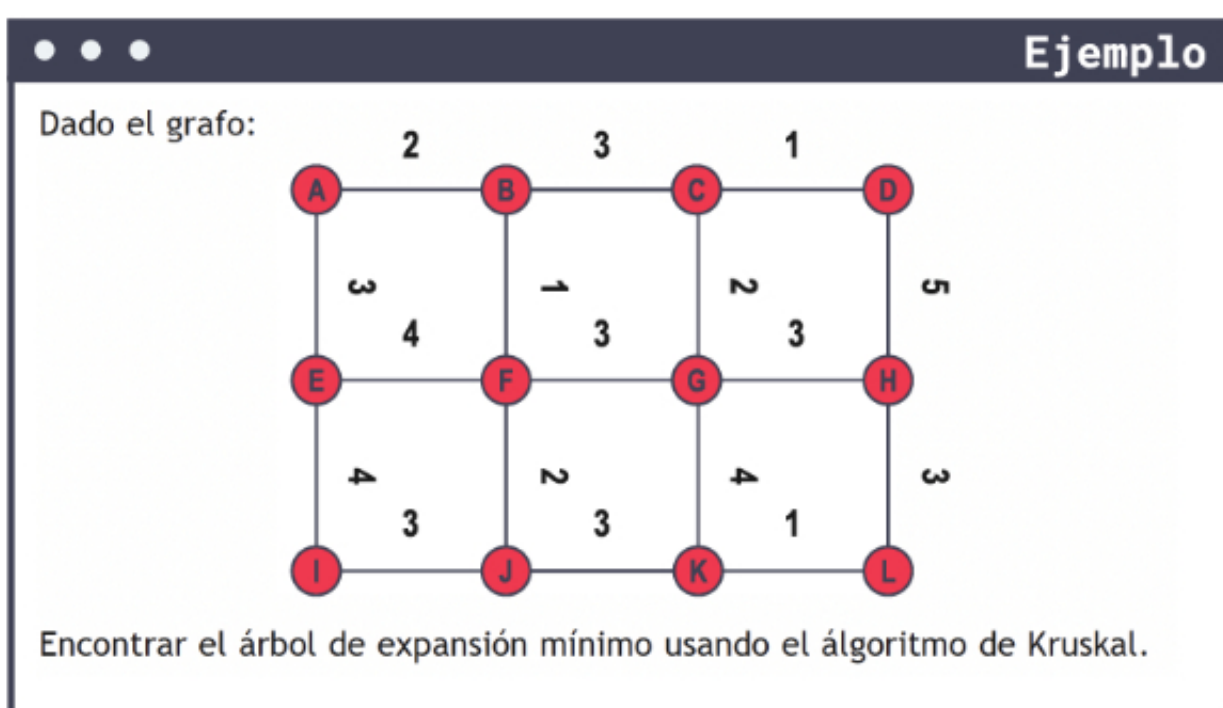
Existen varios algoritmos para encontrar el árbol de expansión mínimo de un gráfico, que incluyen:

El algoritmo de Kruskal es un algoritmo ampliamente utilizado para encontrar el árbol de expansión mínimo de un gráfico no dirigido conectado y con pesos.

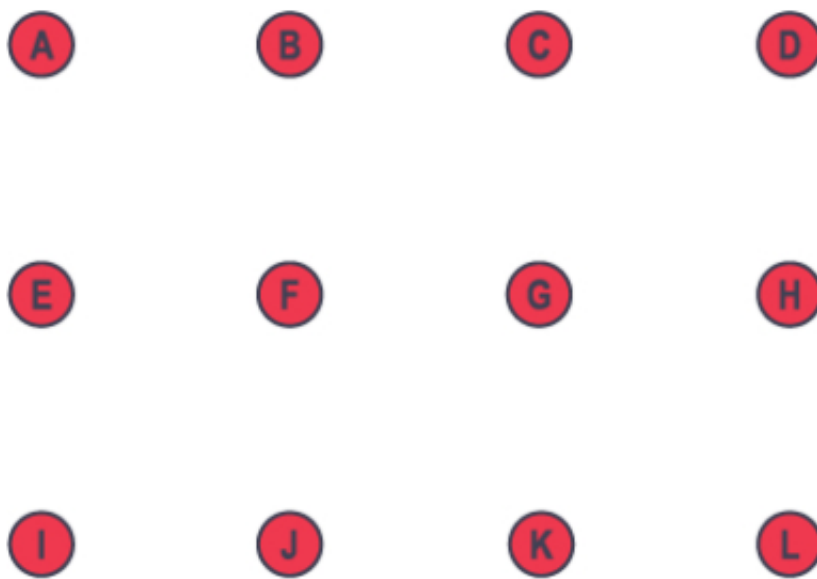
Este algoritmo comienza con un conjunto vacío de aristas y agrega iterativamente la siguiente arista de menor peso que no forma un ciclo hasta que se incluyen todos los vértices.

Para entender un poco más como funciona, se debe hacer lo siguiente:

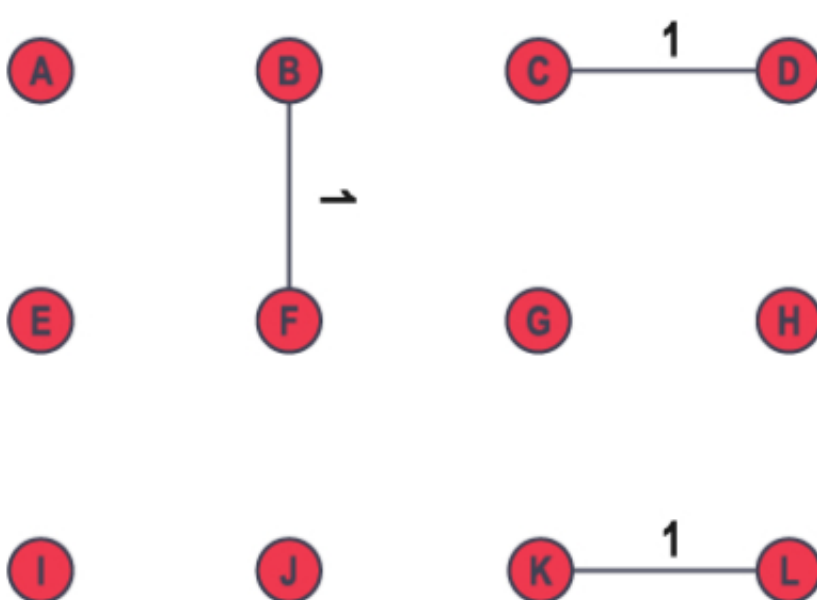
- Ordenar todos los lados en orden creciente de su peso.
- Crear un conjunto vacío de lados que, eventualmente, formarán el árbol de expansión mínimo.
- Iterar a través de los lados ordenados, agregando cada lado al conjunto del árbol de expansión mínimo si y solo si no crea un ciclo en el conjunto del árbol de expansión mínimo que se ha formado hasta el momento. Para verificar si un lado forma un ciclo, se utiliza una estructura de datos de búsqueda de unión para mantener los componentes conectados del gráfico.
- El algoritmo continúa agregando aristas al conjunto del árbol de expansión mínimo hasta que todos los vértices están conectados. Al final, el conjunto del árbol de expansión mínimo que se forma será el árbol de expansión mínimo del grafo.



Se parte del conjunto vacío del árbol de expansión mínimo (AEM), es decir, el AEM es:

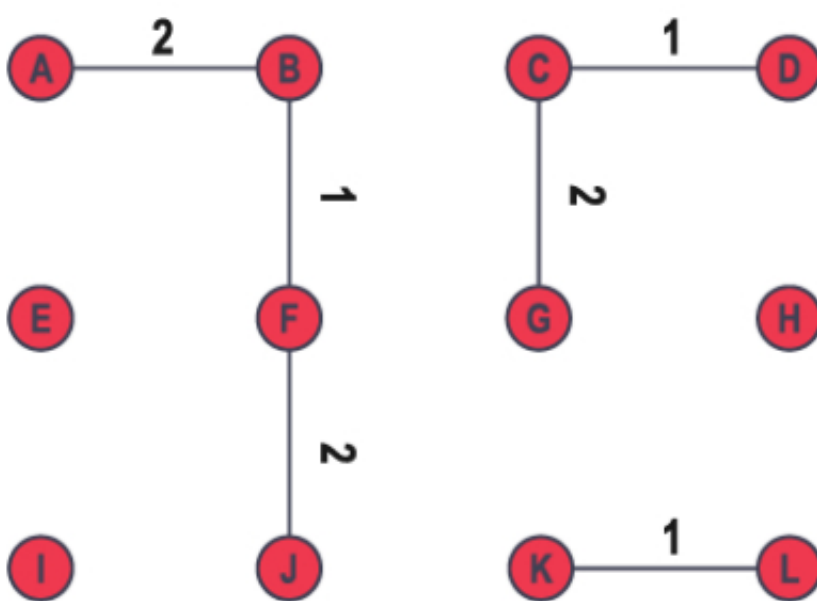


Se hace el conjunto con los pesos en orden creciente, se deja al estudiante formarlo, pero se irán añadiendo los lados de menor peso, evitando los ciclos. Primero los de peso 1. Así el grafo queda:

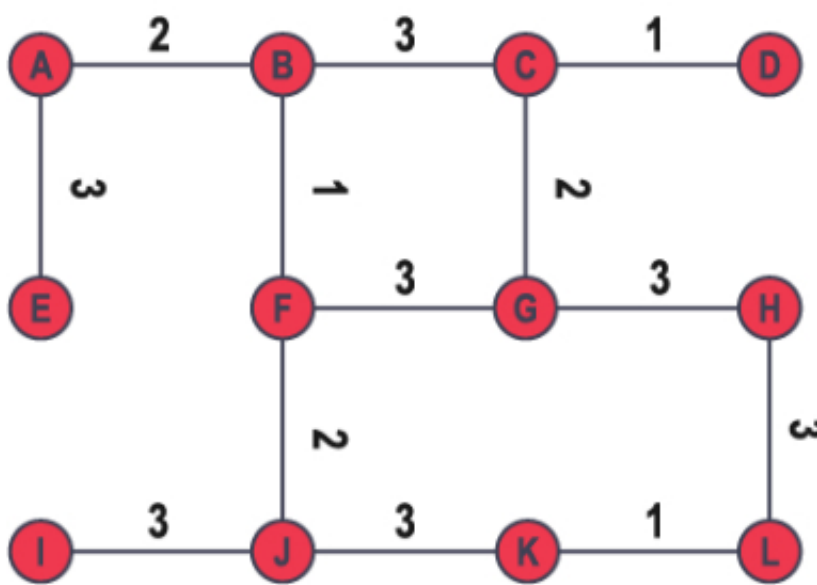


Note que en el grafo no hay ciclos, por lo tanto, estos tres lados quedan en el árbol de expansión mínimo.

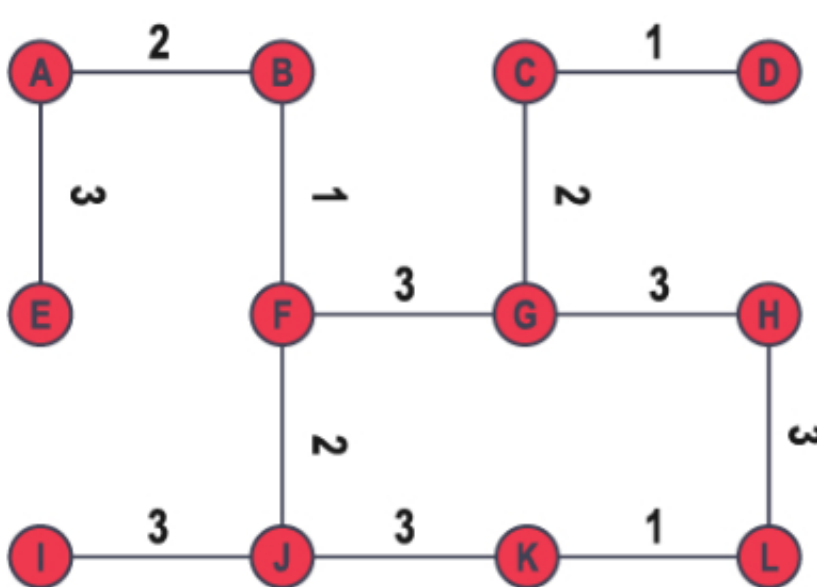
Ahora los de peso 2:



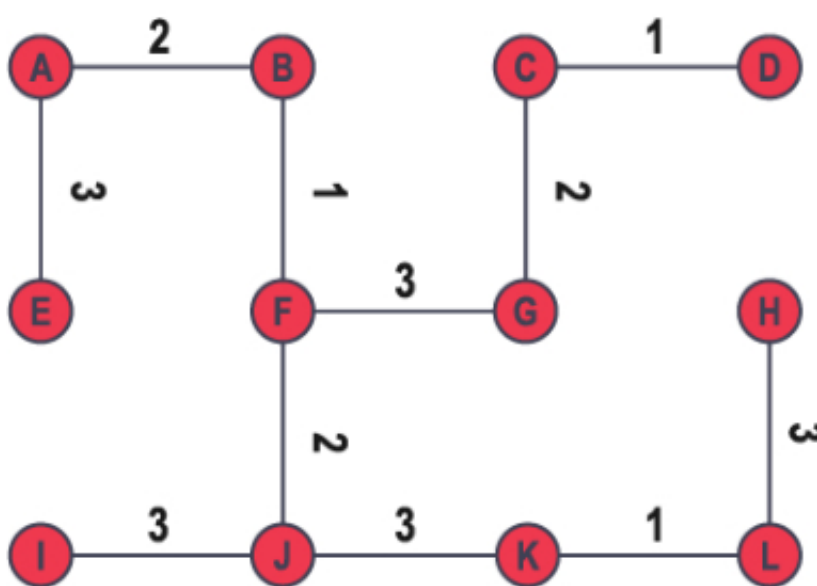
De nuevo no hay ciclos, así que se adicionan los de peso 3:



Se forman dos ciclos. Recuerde que el grafo debe ser conectado para que forme un árbol. Del ciclo *BCFG* se quita uno de los lados de peso 3: *BC*:



Ahora en el ciclo *FGHKJI* se quita *GH*:



Así el AEM es el árbol anterior y la suma de este es 24.

Otro algoritmo importante es el Algoritmo de Prim

Algoritmo de Prim

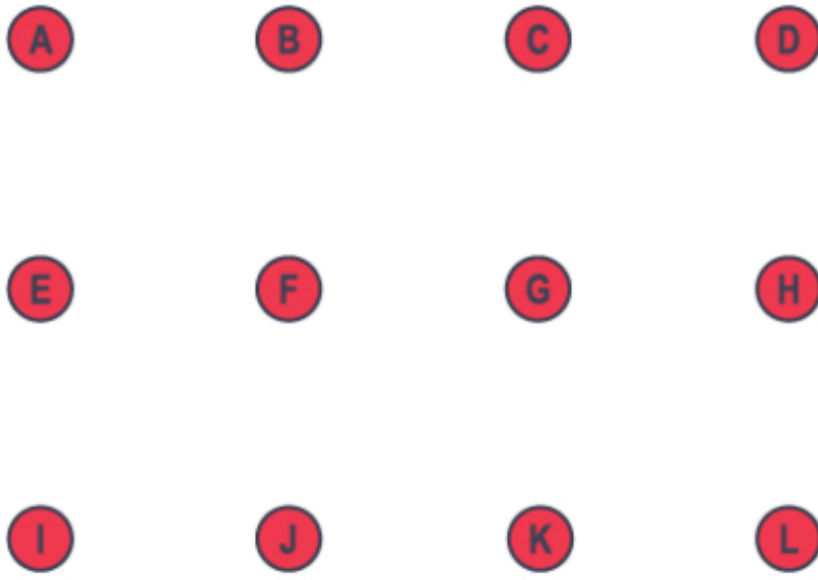
El algoritmo de Prim es otro algoritmo popular para encontrar el árbol de expansión mínima (AEM) de un grafo no dirigido con pesos y conectado.

El algoritmo de Prim funciona de la siguiente manera:

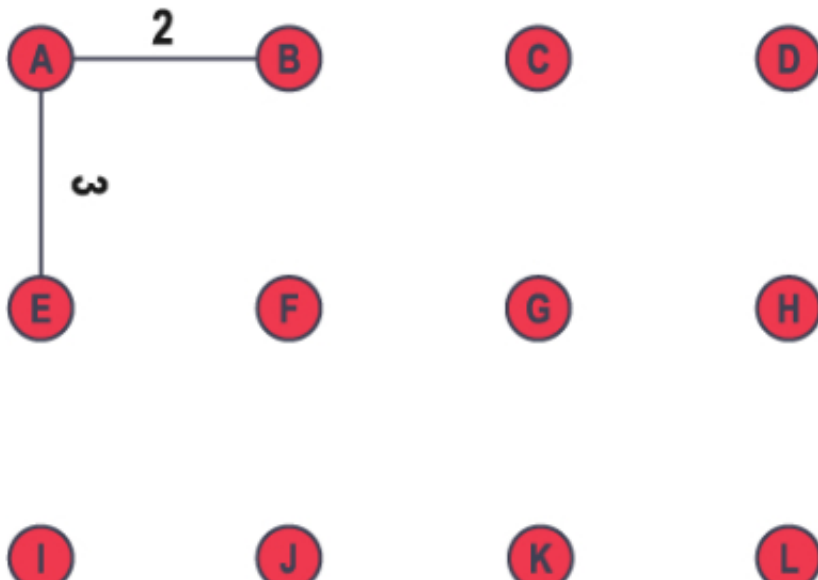
- Se selecciona cualquier vértice del grafo y lo agrega a un conjunto de vértices visitados.
- Se encuentra la arista de menor peso que conecta un vértice visitado con un vértice no visitado. Se agrega el vértice no visitado y la arista al árbol.
- Se repite el paso 2 hasta que todos los vértices del grafo estén en el árbol.

El algoritmo de Prim es similar al de Kruskal en que se construye el AEM de manera secuencial creciente. Sin embargo, en lugar de ordenar todas los lados primero, el algoritmo de Prim comienza con un solo vértice y selecciona las aristas de menor peso que conectan ese vértice con los restantes.

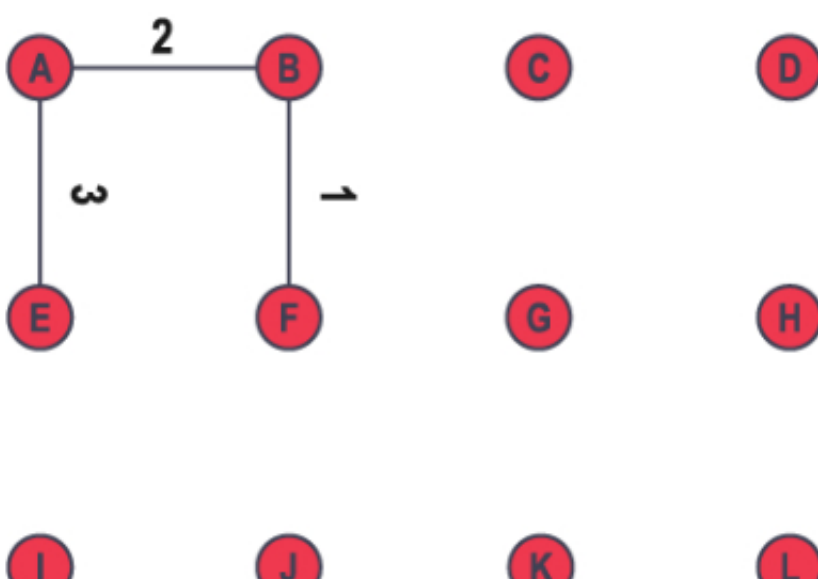
Para el ejemplo anterior, se usa el algoritmo de Prim, desde cero:



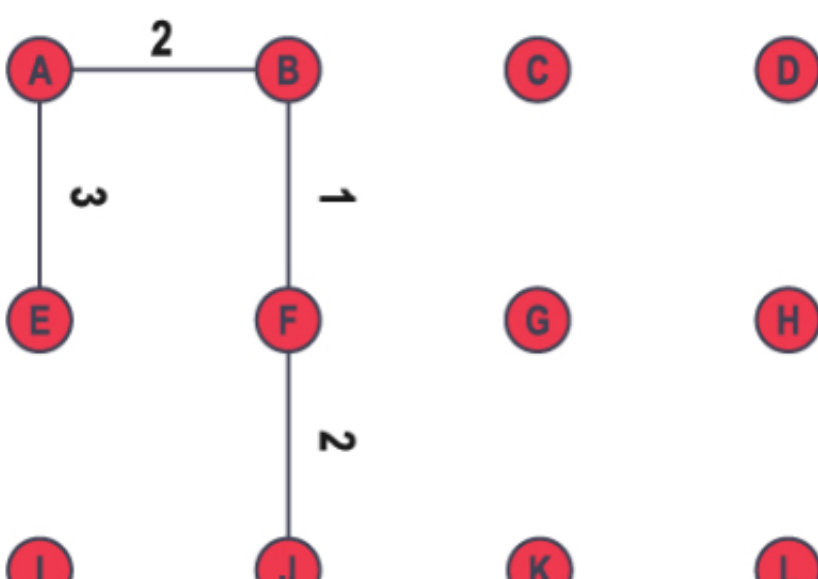
Y se selecciona el vértice A y sus lados adyacentes:



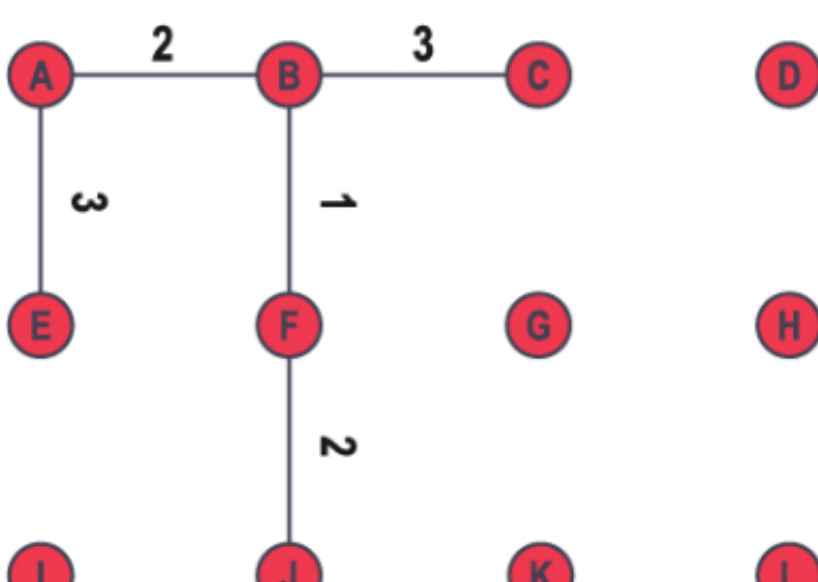
Ahora el lado con el vértice visitado con menor peso es el lado BF , el cual se agrega:



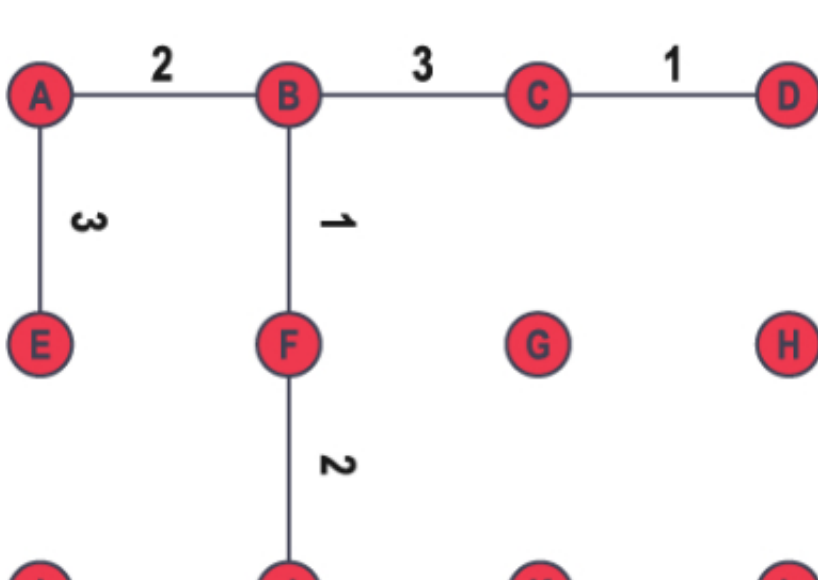
Ahora se agrega el lado FJ :



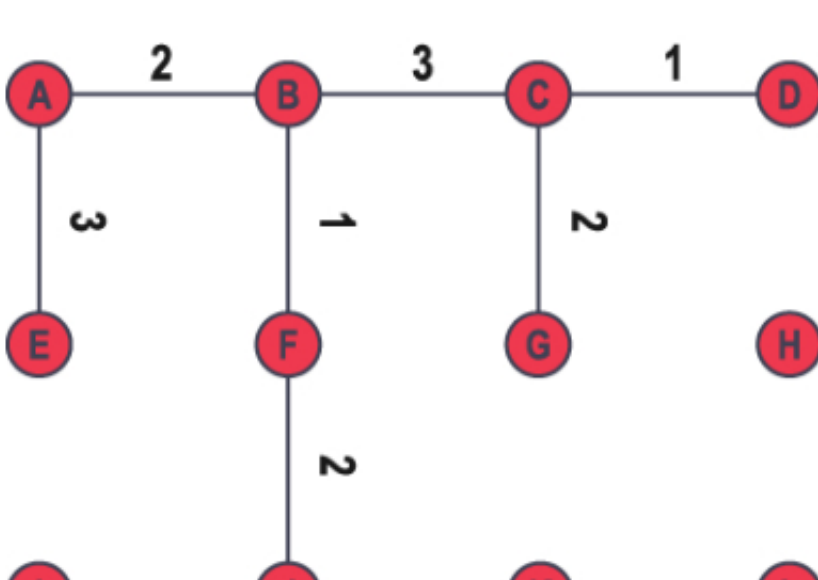
Ahora se agrega BC :



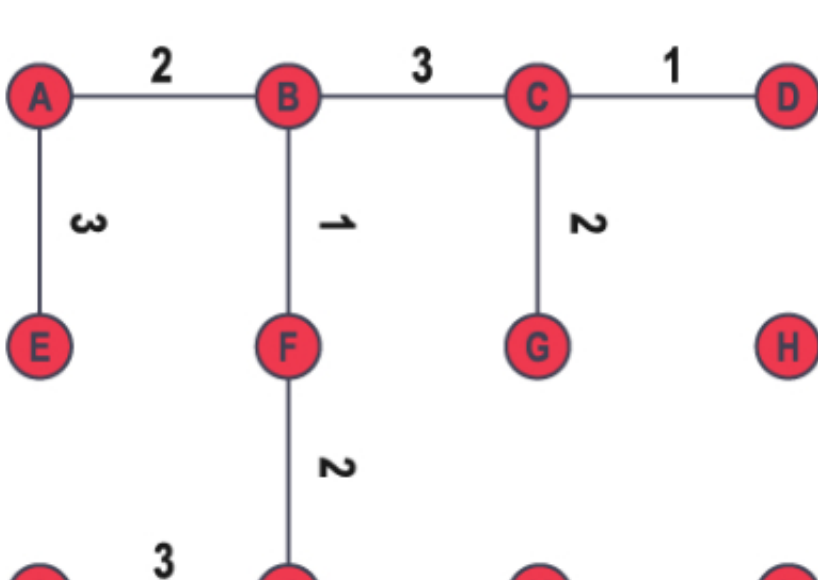
Se agrega ahora CD :



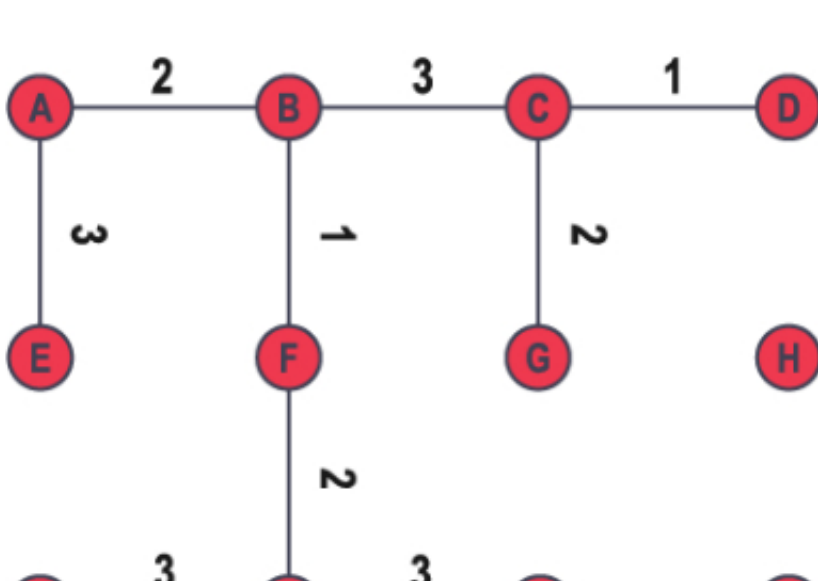
Se agrega CG :



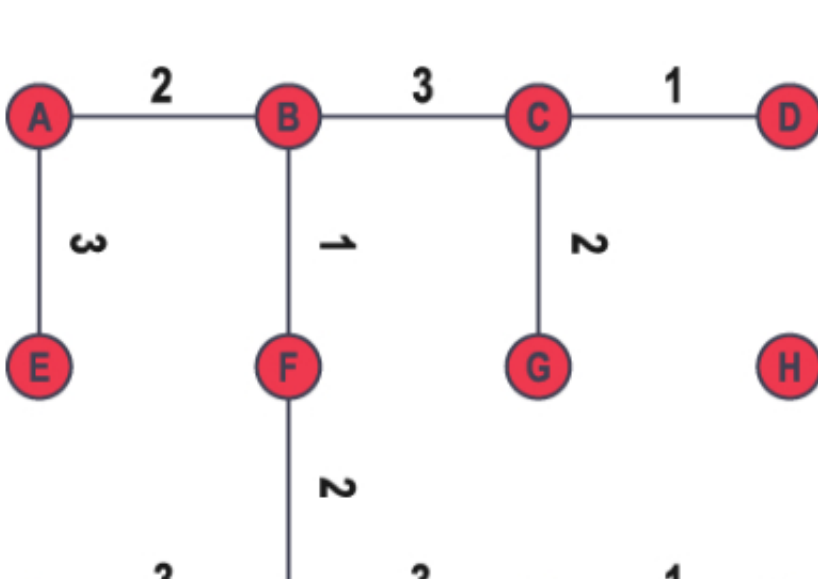
Se agrega IJ :



Ahora se agrega JK :



Ahora se agrega KL :



Se agrega HL :

Lo cual forma el AEM y tiene peso 24.

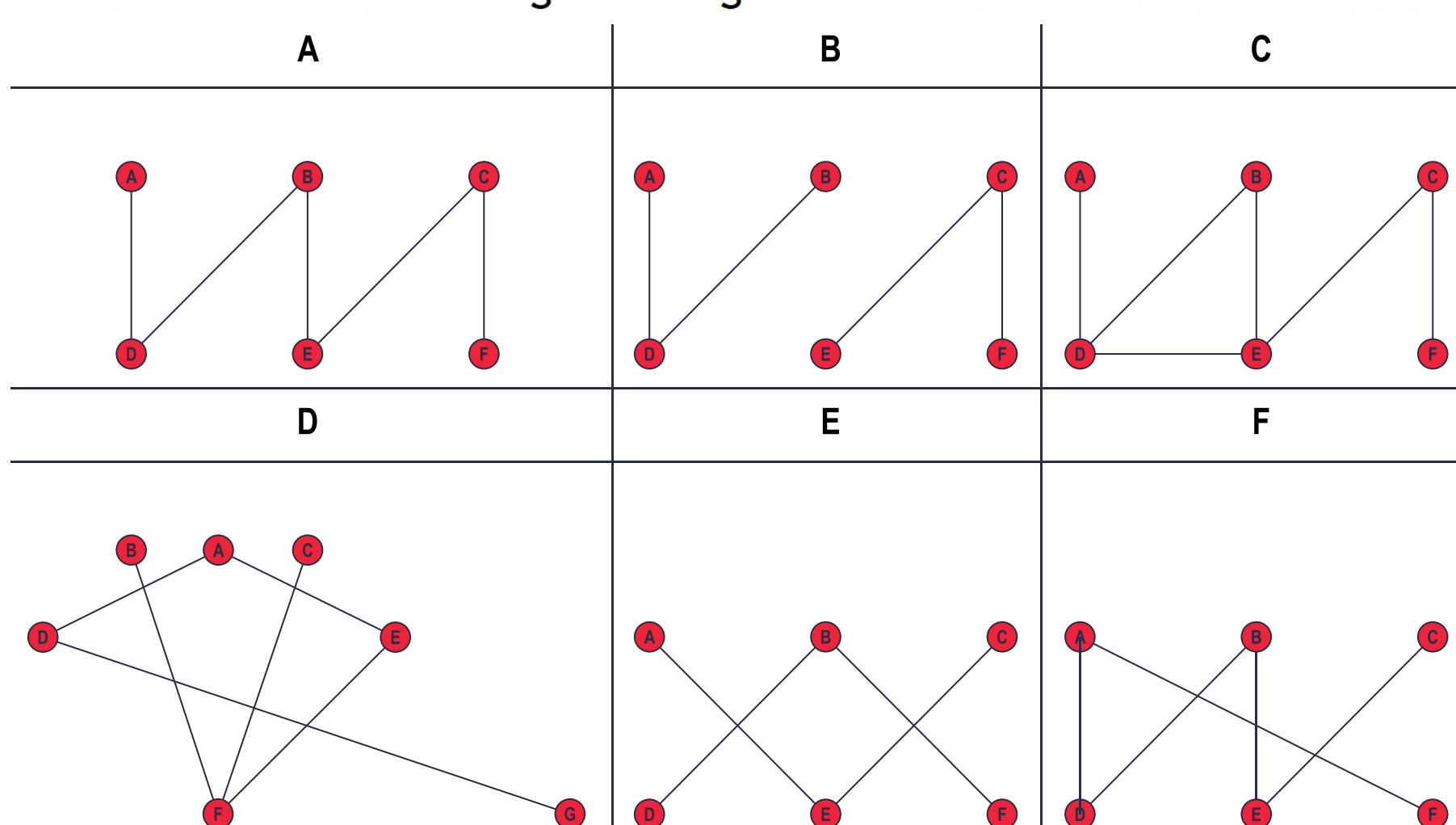
Pero note que en el algoritmo de Prim se puede obtener un AEM más grande debido a que depende del vértice de donde se inicia.

El árbol de expansión mínimo tiene aplicaciones en el diseño de redes, la planificación del transporte y otras áreas en las que es importante minimizar el costo de conectar diferentes nodos.

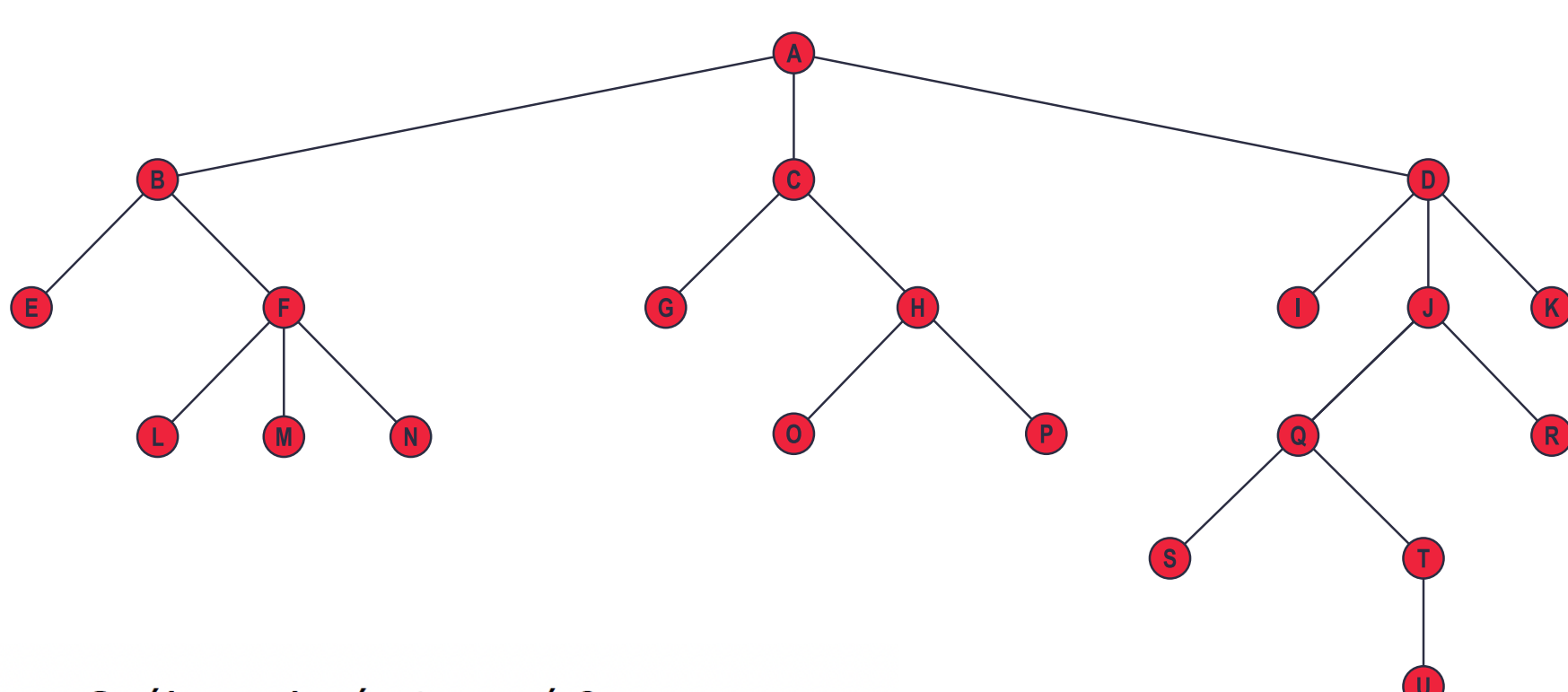
Árboles, algoritmo Kruskal y de Prim

Ejercicios

1. Determine cuáles de los siguientes grafos son árboles:



2. Dado el árbol enraizado:



- ¿Cuál es el vértice raíz?
- ¿Cuáles son los vértices internos?
- ¿Cuáles son las hojas?
- ¿Cuáles son los hijos de f ?
- ¿Cuál es el padre de h ?
- ¿Cuales son los hermanos de m ?
- ¿Cuáles son los ancestros de p ?
- ¿Cuáles son los descendientes de b ?

3. ¿Cuántos árboles no isomorfos existen con 5 vértices?
4. ¿Cuántos árboles no isomorfos existen con 5 vértices si se usan direcciones?
5. Construya un árbol completo binario de altura 4.
6. Construya un árbol m -ario completo con 76 hojas y altura 3, donde m es un entero positivo o demostrar que no existe ese árbol.
7. Una carta en cadena comienza cuando una persona envía una a 5 personas. Cada persona que recibe la carta tiene la opción de enviarla a otras 5 que nunca la hayan recibido o no enviarla a nadie. Suponga que 4567 personas envían la carta antes de que termine la cadena y que nadie recibe más de una. ¿Cuántas personas reciben la carta y cuántas no la envían?
8. Suponga que 1246 personas participan en un torneo de ajedrez. Usando un modelo de árbol con raíz para el torneo, determine cuántos juegos deben jugarse para encontrar un campeón, si un jugador es eliminado después de una derrota y los juegos se juegan hasta que solo quede un participante que no haya perdido (suponga que no hay empates).
9. Un árbol de búsqueda binaria es un árbol enraizado que se construye a partir de una secuencia de números o caracteres que tengan un orden. El árbol tiene como raíz el primer número de la secuencia, los demás números se ponen siguiendo la regla de que si es menor se pone a la izquierda del nodo actual, si es mayor a la derecha. En un árbol de búsqueda binario todo subárbol es un árbol de búsqueda binario.

A partir de esta definción construya los árboles de búsquedad binarios de las siguientes sucesiones:

- a) $l_1 = \{58, 100, 49, 79, 80, 62, 14, 73, 42, 56, 70, 64, 46, 10, 18, 21, 69, 36, 20\}$
- b) $l_2 = \{99, 17, 60, 76, 57, 20, 19, 12, 27, 25, 83, 13, 61, 66, 25, 48, 22, 35, 51, 17\}$
- c) $l_3 = \{25, 71, 88, 18, 69, 66, 64, 42, 40, 72, 56, 45, 71, 46, 12, 69, 66, 91, 20, 55\}$
- d) $l_4 = \{82, 89, 30, 45, 24, 67, 40, 55, 41, 31, 11, 55, 15, 44, 81, 66, 74, 73, 71, 44\}$

10. En el ejercicio anterior determine la altura de cada árbol.
11. En el ejercicio 1 se dice que un vértice a es un ancestro de otro vértice b . Si existe un camino creciente desde b hasta a , determine todos los ancestros de:
 - a) 14 en l_1
 - b) 40 en l_2
 - c) 71 en l_3
 - d) 11 en l_4
12. ¿Cuántos árboles de expansión tiene cada uno de los diferentes grafos simples?
 - a) K_3
 - b) K_4
 - c) $K_{2,3}$
 - d) C_5